1. Introduction

1.1 Problem Statement

In today's digital economy, e-commerce has become a necessity rather than a luxury. However, a significant challenge exists for small and medium-sized vendors who wish to sell their products online. Building and maintaining a dedicated e-commerce website requires substantial investment in terms of money, time, and technical expertise — resources that most small vendors simply do not have access to.

Existing large-scale marketplace platforms such as Amazon or Flipkart, while popular, charge high commission rates on every transaction, impose strict listing policies, and provide vendors with very limited control over their own brand identity and customer relationships. Independent vendors are left with no direct access to their customer data, no customization of their storefront, and heavy dependency on a third-party platform.

This project addresses the above problem by proposing a custom-built **E-Commerce Multi-Vendor Platform** — a shared marketplace where multiple independent vendors can register, set up their own digital shop, list products, and manage orders through dedicated dashboards. Customers can browse products from all vendors in a single unified interface, add items to a cart, and place orders with ease. A platform administrator oversees vendor approvals, manages product categories, and monitors overall platform activity.

- **Simulated Payments:** Payment operations are mock-only (Cash on Delivery / simulated checkout), without a live payment gateway such as Razorpay or Stripe.
- **Local Media Storage:** Product images are uploaded and served from the local filesystem (`backend/media/products/`) via Django's `MEDIA_ROOT` and Pillow. Cloud-based CDN storage (e.g., AWS S3) is reserved for production deployment.
- **Client-Side Cart:** The shopping cart is maintained exclusively in React Context API memory and is cleared on browser refresh or logout. Server-side persistent cart is a future enhancement.
- **Small-Scale Deployment:** The infrastructure uses SQLite for development storage. PostgreSQL migration is supported via a single `settings.py` config change.

1.2 Abstract

The rapid growth of online commerce has created a strong demand for multi-vendor marketplace platforms, where multiple independent sellers can operate under one unified digital storefront. However, affordable and customizable solutions for small-scale vendors remain scarce, pushing them toward expensive third-party platforms with limited flexibility.

This project presents the design and development of a full-stack E-Commerce Multi-Vendor Platform built using **Django REST Framework (DRF)** as the backend API layer and **React.js** as the frontend single-page application (SPA). The platform implements a three-role user system: Admin, Vendor, and Customer — each with a dedicated dashboard and role-specific access controls enforced through JWT (JSON Web Token) authentication.

Vendors can register on the platform, await admin approval, and then manage their product listings (with image uploads supported via Pillow), track incoming customer orders, and monitor their sales performance through a vendor dashboard. Customers can browse all available products across vendors, filter by category, search by keyword, add items to a dynamic cart, and place orders with a delivery address. The Admin can approve or revoke vendor accounts, manage product categories, manage all users and orders, and view platform-wide statistics including total users, vendors, customers, products, orders, pending orders, and total revenue.

The backend is developed using Django and Django REST Framework, with SQLite for development storage and full PostgreSQL compatibility for production. SimpleJWT handles secure token-based authentication with a 60-minute access token lifetime and 7-day refresh token rotation. The frontend is built with React 18 (via Vite), using React Router v6 for client-side navigation across 20 distinct page components, Axios for API communication with automatic JWT header injection and silent token refresh interceptors, and the Context API for global state management of authentication state and shopping cart data.

The expected outcome is a fully functional multi-vendor marketplace demonstrating real-world information system development concepts including REST API design, role-based access control (RBAC), component-based UI architecture, relational database modeling, and end-to-end integration between a modern JavaScript frontend and a Python-based backend.

Keywords: Multi-Vendor E-Commerce, Django REST Framework, React.js, JWT Authentication, Role-Based Access Control, REST API, SQLite, Single-Page Application.